Web Camera Server Based on Raspberry Pi 4

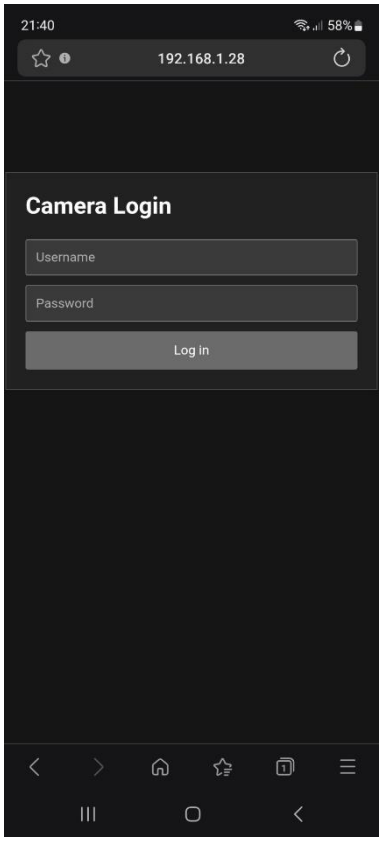
1. User Perspective

Overview:

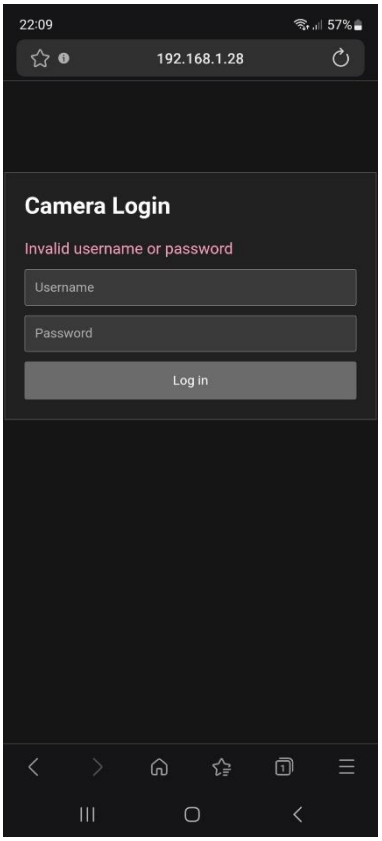
The system provides a web-based interface, which allows a single authorized user to view the live video stream coming from a web camera, attached to a Raspberry Pi 4 board. The system is designed to be used by a nontechnical person and easily restores its functionality, if the hardware fails or disconnects.

System Usage & Behavior:

Authentication: The user accesses the system via a web browser by entering Raspberry Pi's IP address on port 5001. The “Camera Login” form is a simple login screen (pic1). If the user enters incorrect credentials, the system displays a red error message in the form (pic2).



Pic 1.



Pic 2.

Pic 3.

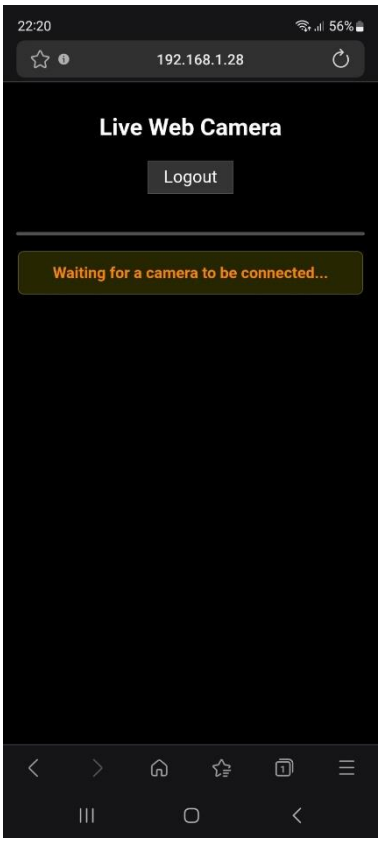
Live Viewing: When the user enters the correct credentials, he is redirected to the “Live Web Camera” page. At the center of the page, there is a video player window, displaying the live feed from the camera (pic3).

The system provides feedback messages below that window, indicating the status of the camera. If the hardware fails, a red warning box appears stating the specific error (e.g., "Error: Camera disconnected").

If the camera is unplugged, the yellow message “Waiting for a camera to be connected” appears bellow the “Logout” button (pic 4). If the user plugs in the camera, after a while the video player window appears, with the green success message “Camera connected”, that fades out after 3 seconds (pic 5).

If the user unplugs the camera while streaming, a yellow warning message “Camera disconnected. Waiting for reconnection” appears (pic 6). If the user reconnects the camera, the green success message appears again (pic 5).

Any time clicking on the “Logout” button, the system releases the camera hardware and returns the user to the “Camera Login” form (pic 1).



Pic 4.

Pic 5.

Pic 6.

2. Designer Perspective

Overview:

The system is built using a Python/Flask backend and a vanilla HTML/JS/CSS frontend. It utilizes OpenCV for video processing, a single-threaded server architecture with a thread-safe camera wrapper, and streams video using the MJPEG protocol alongside asynchronous JSON polling for UI updates.

System Behavior & Architecture:

Web Application Framework: The backend is a Flask application running on 0.0.0.0:5001. Route access (for viewing the HTML, the video stream, and the error endpoint) is strictly controlled via Flask session variables.

Video Capture & Processing: Video is captured using OpenCV (cv2.VideoCapture) specifically forcing the cv2.CAP_V4L2 (Video4Linux2) backend.

The CameraStream class acts as a thread-safe singleton. It utilizes a threading.Lock() to prevent race conditions between the continuous frame-reading loop and the periodic client requests checking for hardware errors.

Streaming Mechanism:

Video is served via the /stream endpoint using a multipart HTTP response (multipart/x-mixed-replace; boundary=frame).

A generator function (generate_mjpeg()) continuously yields JPEG-encoded frames directly to the browser's <img> tag.

Client-Side Architecture (Frontend):

The frontend avoids complex WebSocket implementations. Instead, the client executes an AJAX fetch request to the /camera_error endpoint every 2000ms.

This JSON payload contains the current hardware state, which JavaScript uses to toggle CSS classes (show, fade-out) on the notification elements.

Fault Tolerance & Auto-Discovery:

The system includes a transient error buffer (_max_read_failures = 2) to prevent minor frame drops from triggering a full hardware reset.

If the camera is completely lost, the capture object is released. The system enters a polling loop, waiting 1 sec (_RECONNECT_DELAY) before re-attempting to open the /dev/video0 node.

Graceful shutdown is handled by catching SIGINT, ensuring the hardware is released when the server terminates.

3. Constraints

Hardware & Environment:

Host Device: Requires a Raspberry Pi 4 (or a Linux environment natively supporting V4L2).

Camera: A standard USB UVC-compatible camera.

Network: The server runs on HTTP over port 5001. Port forwarding or local network access is required.

Performance Constraints:

Resolution & Framerate: Hardcoded to 640x480 resolution. The software pacing logic targets a maximum of 30 frames per second (_FRAME_INTERVAL = 1.0 / 30).

Compression: Frames are encoded to JPEG with a quality parameter of 80 to balance visual fidelity with bandwidth limits.

Security Constraints:

Authentication: The system currently utilizes a single, hardcoded set of plaintext credentials, making it unsuitable for public internet deployment without a reverse proxy or VPN.

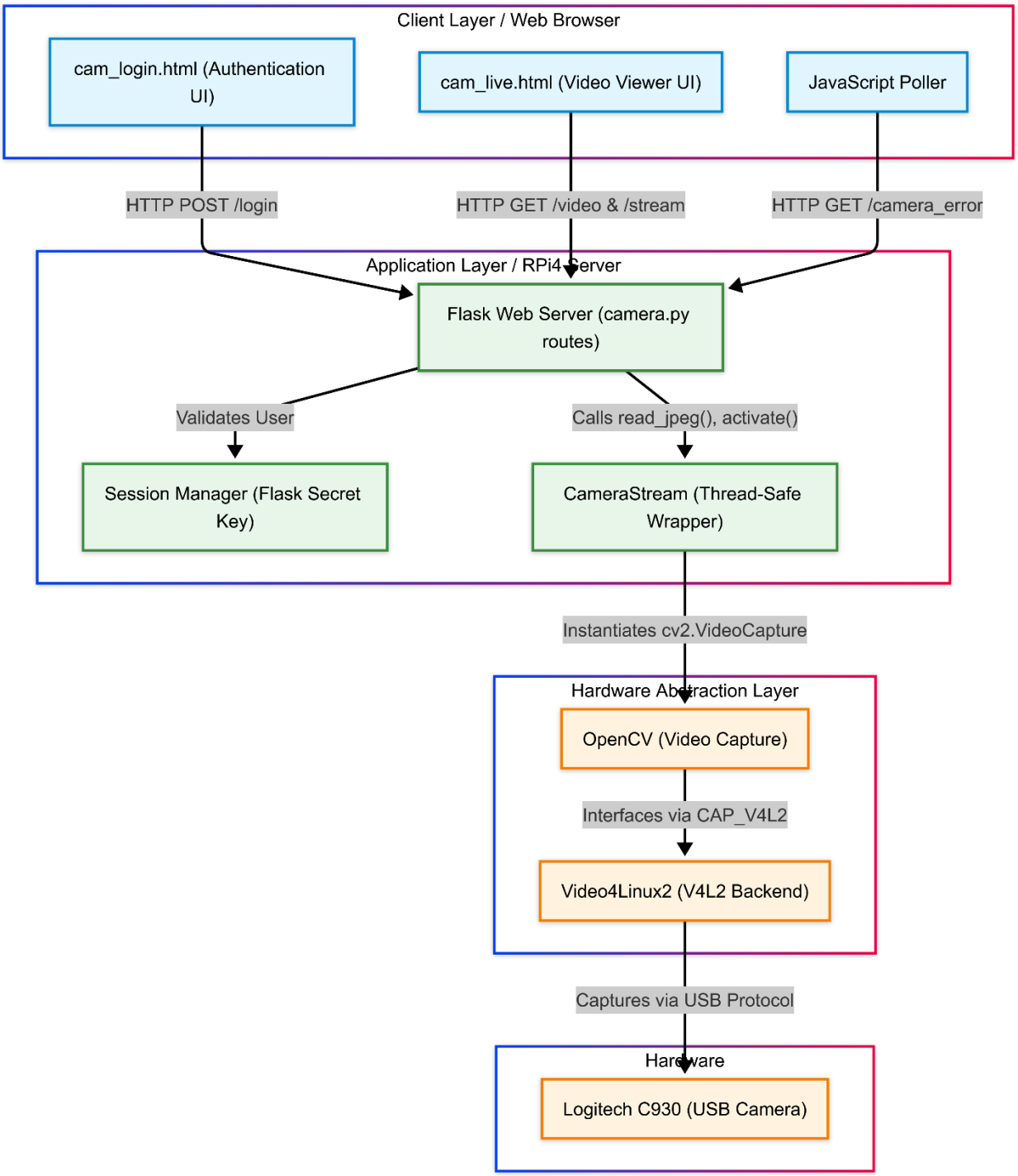
Session Management: The Flask secret key ("secret_password_for_sessions") is hardcoded directly into the script.

Protocol: Traffic is unencrypted HTTP, meaning the video stream and login credentials can be intercepted on a local network.

Cost Constraints:

Hardware: ~$50–$80 for the Raspberry Pi 4 and the cost of the chosen USB camera.

Software: $0 (Utilizes free, open-source libraries: Python, Flask, OpenCV).



System Component Diagram